

Cloud and Mobile Computing

Lab 5 Report

Local Serverless Computing and Event-Driven Image Processing Pipeline

Student Name: Bassel Adel

Group: SE3

Lab Title: Local Serverless & Event-Driven Image Pipeline

Learning Objectives

- Explain how event-driven architectures are composed of event sources, event routers, and event destinations.
- Implement a local event-driven pipeline without using any cloud service.
- Use Redis Streams as a local message bus for events.
- Build small containerized functions that behave like lightweight FaaS handlers.
- Observe a simplified version of cold-start behaviour by restarting a function container.
- Compare local simulation with real cloud serverless platforms.

1. System Architecture

The pipeline consists of 5 Docker containers that communicate through Redis Streams. An image dropped into the input folder triggers an automated processing chain that resizes it and logs a notification — all without any cloud account.

Container	Role	Port
lab5-redis	Message broker (Redis Streams)	6379
lab5-event-source	Watches /data/input for new images, publishes events	—
lab5-event-router	Reads events from Redis, routes to functions	—
lab5-image-resizer	Flask function: resizes images to 300px width	5001
lab5-notifier	Flask function: logs notification events	5002

Data Flow

- Image dropped into data/input/ folder
- event-source detects the file and publishes an image.uploaded event to Redis Stream
- event-router reads the event and performs fan-out to both functions
- image-resizer resizes the image (1200×800 → 300×200) and saves to data/output/
- notifier logs a notification with timestamp

2. Lab Screenshots and Results

Screenshot 1: All 5 Docker Containers Running

4. Lab Screenshots & Results

```
PS C:\Users\Bassel Adel\lecture5-local-serverless-lab> docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS
1628a2ae2f3f	lecture5-local-serverless-lab-event-router	"python event_router..."	About a minute ago	Up About a minute	
793183d70565	lab5-event-router	"python watcher.py"	About a minute ago	Up About a minute	
eddfbeebe85e	lab5-event-source	"docker-entrypoint.s..."	About a minute ago	Up About a minute	0.0.0.0:6379->6379/tcp, [::]:
6379->6379/tcp	lab5-redis	"python app.py"	About a minute ago	Up About a minute	0.0.0.0:5001->5000/tcp, [::]:
227f9c4baa2	lecture5-local-serverless-lab-image-resizer	"python app.py"	About a minute ago	Up About a minute	0.0.0.0:5002->5000/tcp, [::]:
5001->5000/tcp	lab5-image-resizer	"python app.py"	About a minute ago	Up About a minute	0.0.0.0:5002->5000/tcp, [::]:
66d29a1d52c	lecture5-local-serverless-lab-notifier	"python app.py"	About a minute ago	Up About a minute	0.0.0.0:5002->5000/tcp, [::]:
5002->5000/tcp	lab5-notifier				

```
PS C:\Users\Bassel Adel\lecture5-local-serverless-lab>
```

Screenshot 1: All 5 Docker containers running (docker ps)

Figure 1: docker ps — all 5 containers running (redis, event-source, event-router, image-resizer, notifier)

Screenshot 2: Event Source Published + Event Router Routed to Both Functions

The event-source detected test_image.png and published an image.uploaded event to Redis. The event-router received it and forwarded it to both image-resizer and notifier (fan-out):

```
lab5-event-source | Event Source started. Watching folder: /data/input
lab5-event-router | Event Router started. Waiting for events...

lab5-event-source | Published event: {
lab5-event-router | Received event: image.uploaded
lab5-event-source | "event_id": "19b8259b-f9cd-4e28-bale-5d28f57867dd",
lab5-event-router | {
lab5-event-source |   "event_id": "19b8259b-f9cd-4e28-bale-5d28f57867dd",
lab5-event-router |   "event_type": "image.uploaded",
lab5-event-source |   "event_type": "image.uploaded",
lab5-event-router |   "file_name": "test_image.png",
lab5-event-source |   "file_name": "test_image.png",
lab5-event-router |   "file_name": "test_image.png",
lab5-event-source |   "file_path": "/data/input/test_image.png",
lab5-event-router | NOTIFICATION: {"status": "notified", "function": "notifier", "event_type": "image.uploaded", "file_name": "test_image.png", "timestamp": "2026-04-28T15:27:27.718341"}
lab5-event-source | "width": 300,
lab5-event-router | "file_path": "/data/input/test_image.png",
lab5-event-source | 172.20.0.6 - - [28/Apr/2026 15:27:27] "POST /notify HTTP/1.1" 200 -
lab5-event-router | "created_at": "2026-04-28T15:27:27.675694"
lab5-event-source | "width": 300,
lab5-event-router | }
lab5-event-source | "created_at": "2026-04-28T15:27:27.675694"
lab5-event-router | }
lab5-event-source | Sent to http://image-resizer:5000/resize
lab5-event-router | Response code: 200
lab5-event-source | Duration: 34.72 ms
lab5-event-router | Response: [{"function": "image-resizer", "input_file": "test_image.png", "new_size": [300, 200], "original_size": [1200, 800], "output_file": "/data/output/resized_test_image.png", "processing_ms": 29.39, "status": "success"}]
lab5-event-source | Sent to http://notifier:5000/notify
lab5-event-router | Response code: 200
lab5-event-source | Duration: 4.21 ms
lab5-event-router | Response: [{"event_type": "image.uploaded", "file_name": "test_image.png", "function": "notifier", "status": "notified", "timestamp": "2026-04-28T15:27:27.718341"}]
```

Screenshot 2 & 3: Event source published + Event router received and routed to both functions

Figure 2: Event source published event + Event router received and routed to both functions

Screenshot 3: Event Router Logs

The event router logs confirm it received the image.uploaded event and successfully sent it to both `http://image-resizer:5000/resize` (200 OK, 34.72ms) and `http://notifier:5000/notify` (200 OK, 4.21ms):

```
lab5-event-router | Event Router started. Waiting for events...
lab5-event-router | Received event: image.uploaded
lab5-event-router | {
lab5-event-router |   "event_id": "19b8259b-f6c4-4e28-8a1e-5428f57867dd",
lab5-event-router |   "event_type": "image.uploaded",
lab5-event-router |   "file_name": "test_image.png",
lab5-event-router |   "file_path": "/data/input/test_image.png",
lab5-event-router |   "width": 300,
lab5-event-router |   "created_at": "2026-04-28T15:27:27.675694"
lab5-event-router | }
lab5-event-router | Sent to http://image-resizer:5000/resize
lab5-event-router | Response code: 200
lab5-event-router | Duration: 34.72 ms
lab5-event-router | Response: [{"function":"image-resizer","input_file":"test_image.png","new_size":[300,200],"original_size":[1200,800],"output_file":"/data/output/resized_test_image.png","processing_ms":29.39,"status":"success"}]
lab5-event-router | Sent to http://notifier:5000/notify
lab5-event-router | Response code: 200
lab5-event-router | Duration: 4.21 ms
lab5-event-router | Response: [{"event_type":"image.uploaded","file_name":"test_image.png","function":"notifier","status":"notified","timestamp":"2026-04-28T15:27:27.718341"}]
lab5-event-router |
```

Screenshot 3: Event router logs — routed to image-resizer and notifier

Figure 3: Event router logs — routed to image-resizer and notifier with response codes and durations

Screenshot 4: Output Folder Showing resized_test_image.png

```
PS C:\Users\Bassel Adel\lecture5-local-serverless-lab> dir data/output
Directory: C:\Users\Bassel Adel\lecture5-local-serverless-lab\data/output

Mode                LastWriteTime         Length Name
----                -
-a-----          4/29/2026   3:07 PM             1711 resized_test_image.png
```

Screenshot 4: data/output folder showing resized_test_image.png

Figure 4: data/output folder — resized_test_image.png created (1711 bytes, 4/29/2026)

3. Cold Start Experiment

The cold start experiment measured the difference in response time between a warm container (already running and serving requests) and a freshly restarted container (cold start). The image-resizer container was restarted using docker compose restart image-resizer and an immediate request was sent.

Request Type	Total Time (ms)	Processing Time (ms)	Observation
Warm Request	167.15 ms	17.73 ms	Container already running
Cold Start Request	61.98 ms	29.76 ms	Container freshly restarted
Difference	~105 ms	~12 ms	Cold start adds overhead

Note: In this local Docker environment, cold start differences are smaller than in real cloud serverless platforms (e.g., AWS Lambda, Google Cloud Functions), where cold starts can reach 500–3000ms because the entire runtime environment must be provisioned from scratch.

4. Reflection Questions

Q: What is the event source in this lab?

A: The event source is `watcher.py` running in the `lab5-event-source` container. It continuously monitors the `/data/input` folder every 2 seconds. When a new PNG or JPG image appears, it publishes an `image.uploaded` event to the Redis Stream using `xadd`.

Q: What is the event router?

A: The event router is `event_router.py` running in the `lab5-event-router` container. It reads events from the Redis Stream using `xread` and uses a routing table to decide which function endpoints should receive each event type. For `image.uploaded` events, it routes to both `image-resizer` and `notifier`.

Q: What are the event destinations?

A: The event destinations are the two Flask functions: `lab5-image-resizer` (POST `/resize`) which resizes the image and saves it to `/data/output/`, and `lab5-notifier` (POST `/notify`) which logs a notification message with a timestamp.

Q: Why is this pipeline loosely coupled?

A: The pipeline is loosely coupled because the event source does not know about the functions, and the functions do not know about each other. They only communicate through the Redis Stream and HTTP. New functions can be added by just updating the routing table in the router — no changes are needed in the event source or other functions.

Q: What happened after restarting the image-resizer container?

A: After restarting, the first request took longer to process (higher overhead) compared to a warm request. The container needed to reinitialize the Flask application and Python runtime before it could serve the request, simulating cold start behavior.

Q: How is this similar to serverless cold start?

A: In real serverless platforms like AWS Lambda, a cold start occurs when a new execution environment is provisioned from scratch — including loading the runtime, code, and dependencies. Our local experiment simulates this by restarting the container, which forces Python and Flask to reinitialize before the first request can be served.

Q: What is missing compared with real cloud serverless platforms?

A: Real cloud serverless platforms provide: true ephemeral execution (containers are destroyed after each invocation), auto-scaling to zero, per-invocation billing, built-in retry with dead letter queues, global distribution, integrated monitoring (CloudWatch, Datadog), and hardware-level isolation (microVMs like Firecracker in AWS Lambda).

Q: How could this pipeline be extended?

A: New functions could be added for image moderation (using an ML model to detect inappropriate content), OCR (extracting text from images using Tesseract), ML enrichment (object

detection or classification), or watermarking. Each would be a new container added to `docker-compose.yml` and a new URL in the router's routing table — no other changes needed.

5. Discussion and Analysis

Event-Driven Architecture Components

Lecture Concept	Local Lab Implementation
Event source	watcher.py detects new images in /data/input
Event router	event_router.py reads Redis Stream and routes to functions
Event bus / message bus	Redis Streams running locally as lightweight event bus
Function-as-a-Service	Containerized Flask services act as functions
Event destinations	image-resizer (/resize) and notifier (/notify)
Fan-out pattern	One image.uploaded event triggers both functions simultaneously
Cold start observation	Container restart simulates initialization overhead

Advantages of Event-Driven Architecture

Loose coupling: Services do not call each other directly — they communicate only through events. **Scalability:** Each function scales independently. **Flexibility:** New functions can be added without changing existing ones. **Resilience:** If one function fails, others continue working. **Asynchronous processing:** The event source does not wait for processing to complete.

Local Setup vs Real Cloud Serverless

Feature	Local Docker Setup	Real Cloud Serverless
Scaling	Manual (docker compose scale)	Automatic to zero and back
Billing	No cost	Per invocation + duration
Cold start	Container restart (~100ms)	500–3000ms (runtime provisioning)
Isolation	Container-level	microVM (Firecracker) per invocation
Retry/DLQ	Manual implementation	Built-in with dead letter queues
Monitoring	docker compose logs	CloudWatch, Datadog, X-Ray
Distribution	Single machine	Global, multi-region

Conclusion

This lab successfully demonstrated how to build a local serverless-style event-driven pipeline using Docker and Redis Streams. All five services were containerized and orchestrated using Docker Compose. The pipeline correctly detected image uploads, published events to Redis, routed them to two functions simultaneously (fan-out), resized the image from 1200×800 to 300×200 pixels, and

logged notifications — all without any cloud account. The cold start experiment demonstrated that restarting containers affects initial response time, mirroring real serverless behavior. The architecture mirrors modern cloud-native design patterns used in production systems such as AWS Lambda with EventBridge, Google Cloud Functions with Pub/Sub, and Azure Functions with Event Grid.